

MEMORIA PROYECTO DESPERTADOR INTELIGENTE

Lucas Prado Marqués

Internet De las Cosas

19/06/2026

Índice:

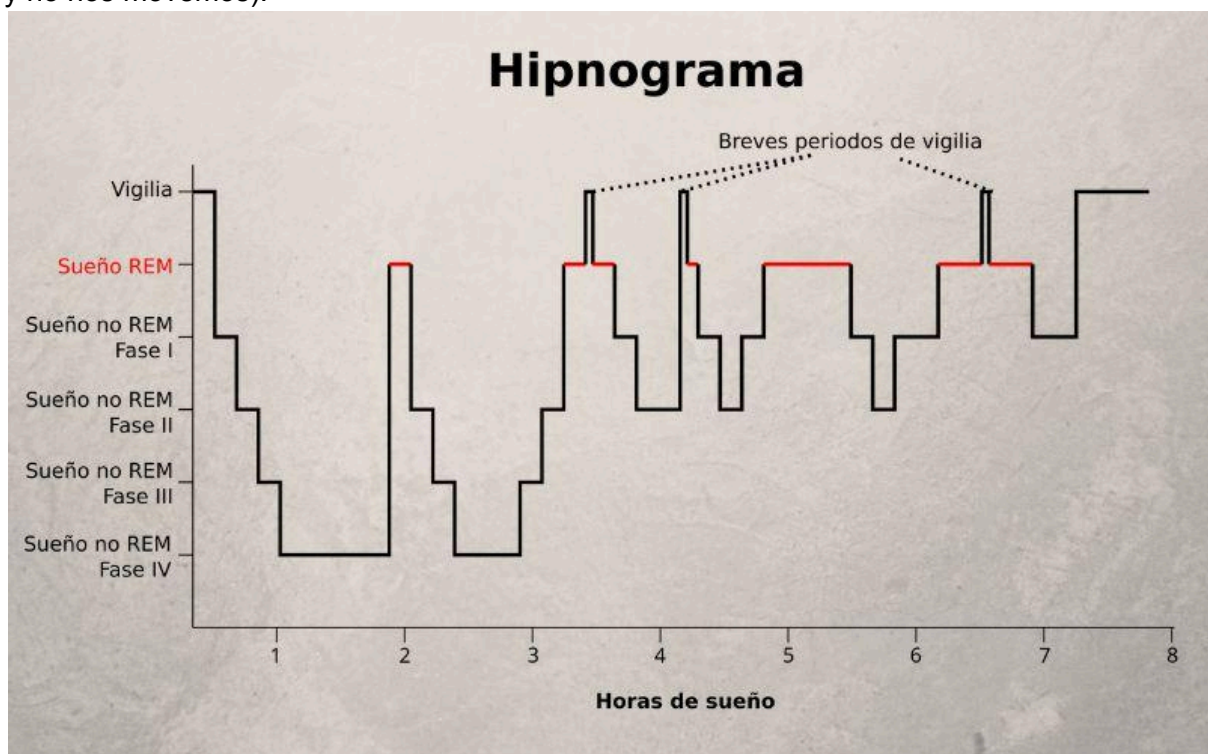
1. Contexto y objetivos del proyecto
2. Trabajo realizado y metodología seguida
3. Resultados obtenidos
4. Problemas encontrados y cómo se resolvieron
5. Conclusiones

Contexto y objetivos del proyecto

Contexto del Proyecto:

Hoy en día, no dormir bien afecta directamente a cómo rendimos durante el día, a nuestro humor y a la salud en general. Uno de los problemas más típicos al madrugar es la famosa inercia del sueño: esa sensación de estar completamente "zombi", pesado y cansado cuando una alarma normal nos despierta de golpe en mitad de una fase de sueño profundo o de una fase REM.

Científicamente, nuestro sueño se divide en ciclos que van cambiando entre fases profundas y ligeras. Si nos despertamos durante el sueño ligero, esa sensación de cansancio disminuye muchísimo, y nos sentimos con más energía desde el primer minuto. La clave está en que, durante el sueño ligero, solemos hacer pequeños movimientos físicos y cambios de postura en la cama. En cambio, en las fases profundas o en la REM, el cuerpo entra en un estado de atonía muscular (básicamente, los músculos se relajan por completo y no nos movemos).



Aprovechando la tecnología actual, podemos solucionar esto usando dispositivos IoT integrados (Edge Computing) que monitoricen el movimiento en tiempo real. Este proyecto nace como una alternativa barata, cómoda y que respeta al 100% la privacidad del usuario, utilizando Inteligencia Artificial embebida para convertir un despertador de toda la vida en un sistema inteligente y personalizado.

Objetivos del Proyecto:

El objetivo principal de este trabajo es diseñar, montar y programar un prototipo de despertador inteligente local. Para ello, utilizaremos hardware de bajo coste y técnicas de TinyML (Machine Learning ejecutado directamente en microcontroladores) para vigilar los movimientos en la cama y elegir el mejor momento para activar la alarma.

Para conseguirlo, nos fijamos los siguientes objetivos específicos:

- **Montaje del Hardware:** Conectar e integrar un sistema electrónico basado en la placa Raspberry Pi Pico (RP2040), un acelerómetro de tres ejes LIS3DH para registrar las vibraciones de la cama y un altavoz pequeño (buzzer) para dar el aviso sonoro.
- **Creación del Modelo TinyML:** Capturar los datos de movimiento a través del protocolo I2C, procesar esas señales mediante análisis espectral y entrenar una red neuronal en la plataforma Edge Impulse. El modelo final debe ser capaz de clasificar en tiempo real si el usuario está en "Sueño Profundo / Calma" o en "Sueño Ligerero / Movimiento".
- **Algoritmo del Despertador (Ventana de Tiempo):** Programar una lógica en el código (una máquina de estados) para definir un rango de tiempo para despertar (por ejemplo, de 7:30 a 8:00). Si la IA detecta que nos movemos dentro de esa media hora, la alarma sonará inmediatamente. Si llegan las 8:00 y no nos hemos movido, el buzzer sonará sí o sí para asegurar que no nos quedemos dormidos.

Trabajo realizado y metodología seguida

Fase 1: Diseño e Interconexión del Hardware

En esta primera etapa del proyecto, me centré en definir el esquema electrónico y realizar el conexionado físico de todos los componentes sobre una placa de pruebas (protoboard) para asegurar que la comunicación entre los dispositivos fuera correcta antes de pasar a la fase de software.

El núcleo del sistema es la Raspberry Pi Pico (RP2040). Para capturar los movimientos de la cama, utilicé el acelerómetro de tres ejes LIS3DH en su versión con conector STEMMA QT, y como alerta sonora integré un módulo de Buzzer pasivo v1.2 de 4 pines.

Decisiones de diseño y conexionado técnico:

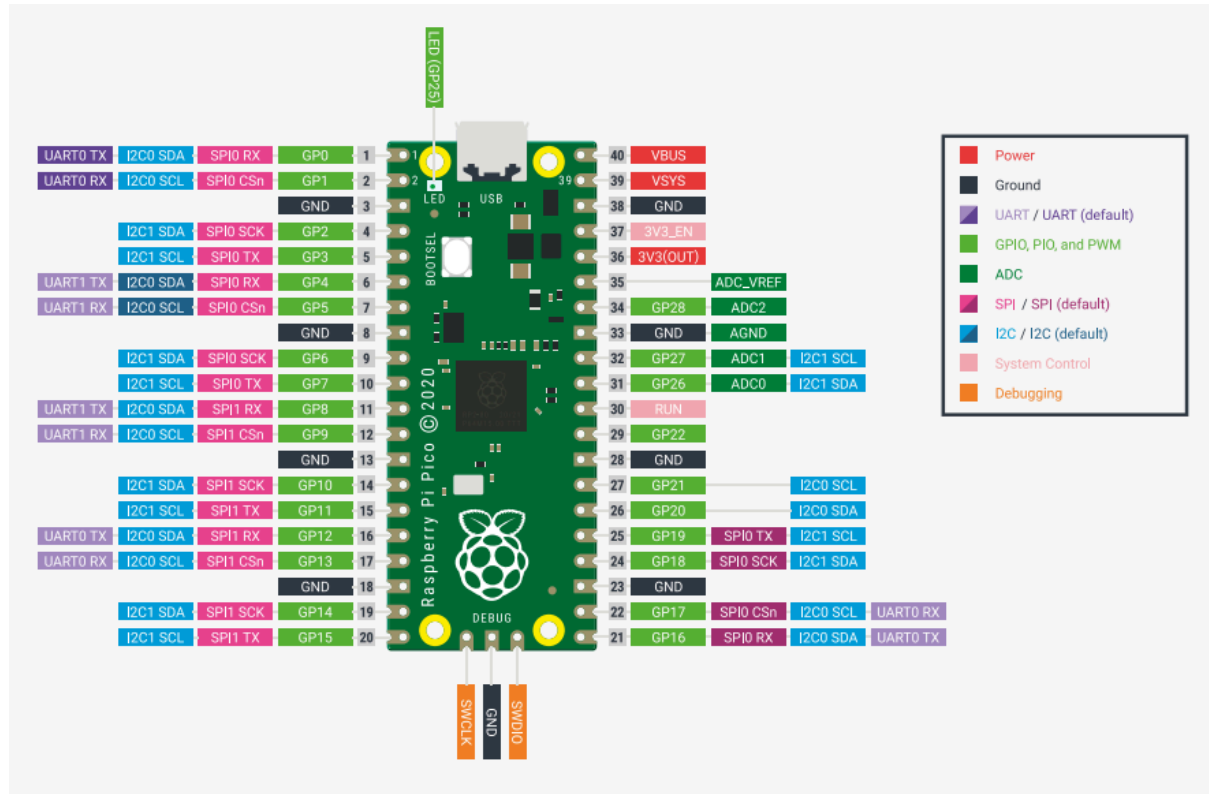
Comunicación I2C para el Acelerómetro: En lugar de usar lecturas analógicas que suelen meter mucho ruido eléctrico, conecté el LIS3DH a través del protocolo I2C. Configuré el bus I2C0 de la Raspberry Pi Pico utilizando los pines GP4 (SDA) para la línea de datos y GP5 (SCL) para la línea de reloj. Al dejar libre el pin de dirección SDO del sensor, este se asignó por defecto en la dirección lógica 0x18.

Gestión de la Alimentación: El acelerómetro funciona a un voltaje lógico de 3.3V, por lo que conecté su pin de alimentación (VIN) directamente al pin 36 (3V3_OUT) de la Pico, compartiendo la misma masa en el pin 38 (GND).

Optimización del Buzzer: El módulo de buzzer v1.2 venía con un cable de 4 hilos (Amarillo, Blanco, Rojo y Negro). Para evitar saturar la línea de 3.3V de la Pico y conseguir que el

sonido del despertador tuviera la máxima potencia posible, decidí conectar el cable de alimentación del buzzer (Rojo) al pin 40 (VBUS), que entrega 5V directos desde el puerto USB. El cable de control de señal (Amarillo) lo conecté al pin GP16 de la Pico para poder modular el tono mediante PWM más adelante, el cable Negro fue a una masa común (GND), y el cable Blanco se dejó desconectado al estar marcado como Not Connected (NC).

Raspberry Pi Pico RP2040



Mapa rápido de conexión de los componentes:



Acelerómetro:

Rojo VIN (3.3V) Pin 36 (Etiquetado como 3V3)

Negro GND (Tierra) Pin 38 (Etiquetado como GND)

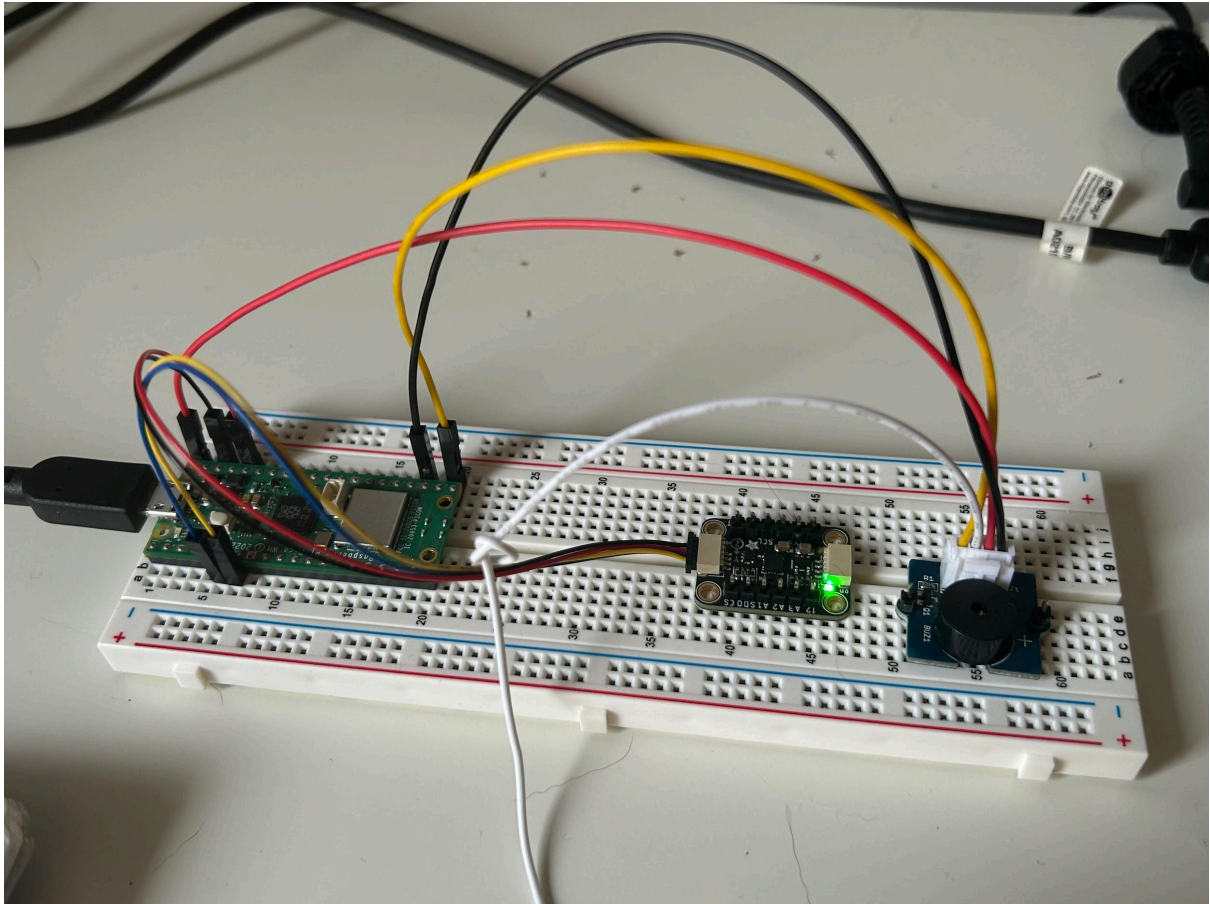
Azul SDA (Datos) Pin 6 (Etiquetado como GP4)
Amarillo SCL (Reloj) Pin 7 (Etiquetado como GP5)



Buzzer:

Rojo (VCC / V) Alimentación (3.3V) Pin 36 3V3 (OUT)
Negro (GND / G) Tierra / Masa Pin 23 (o cualquier GND) GND
Amarillo (SIG / S) Señal de control Pin 21 GP16
Blanco (NC) No conectado No se conecta a nada

Fase 2: Pruebas y Validación del funcionamiento

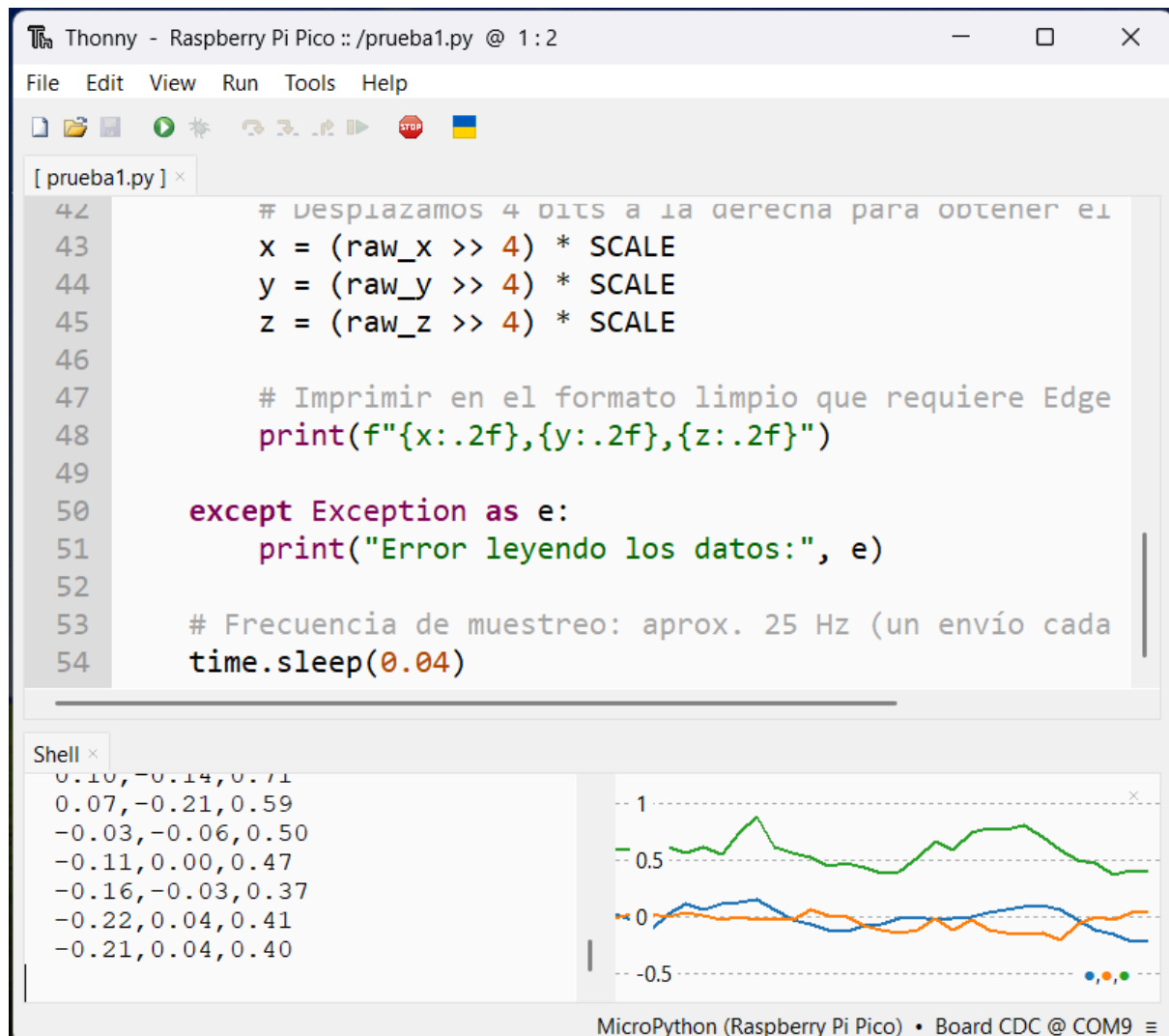


Antes de montar el circuito definitivo en la cama, decidí hacer una fase de pruebas en la mesa para comprobar que todos los componentes se comunicaban bien y que los datos se leían correctamente.

Como se puede ver en la Foto de mi circuito, conecté la Raspberry Pi Pico W, el acelerómetro y el buzzer en la protoboard. En cuanto enchufé la Pico por USB al ordenador, el LED verde del acelerómetro se encendió, lo que me confirmó que el conexionado físico estaba bien hecho y los componentes recibían energía.

Para la parte del software, abrí Thonny y programé un script en MicroPython puro. Decidí hacerlo así, leyendo directamente los registros internos del sensor por I2C, para evitar instalar librerías externas que suelen dar problemas de compatibilidad en la Pico. El código se configuró para pedirle datos al acelerómetro a una frecuencia de 25 Hz, es decir, unas 25 lecturas por segundo, que es el estándar perfecto para captar movimiento humano.

Los resultados en Thonny, tal y como se muestra en la Captura de la pantalla:



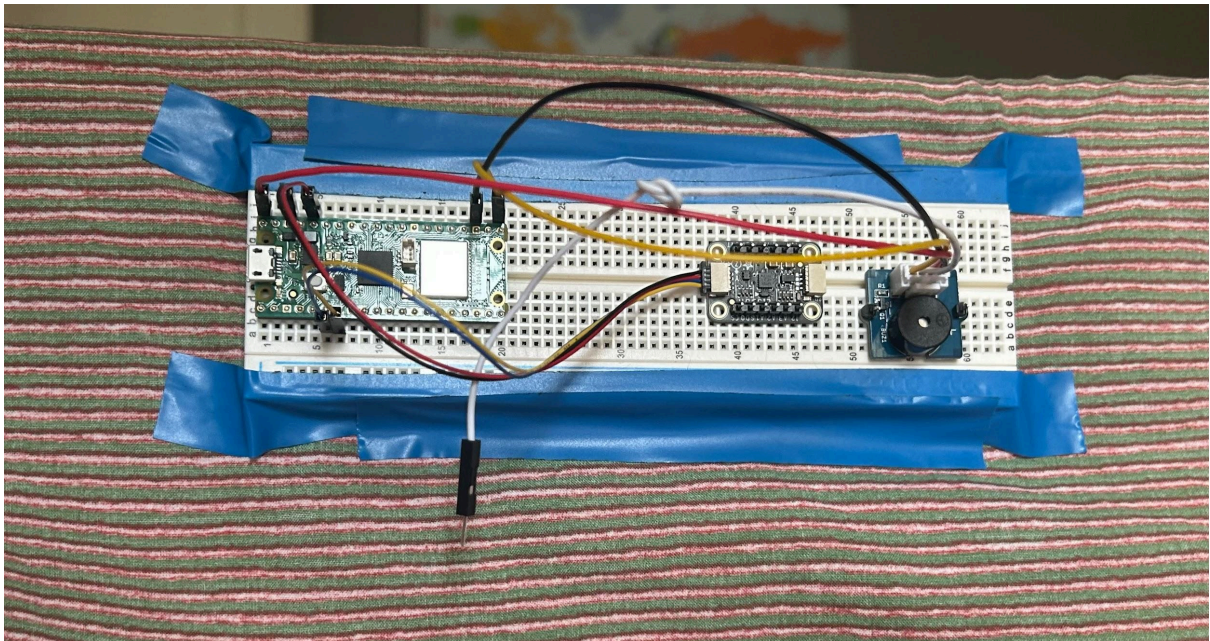
- En la Terminal (Shell): Empezó a salir un flujo continuo de texto con tres números separados por comas. Cada número representa la aceleración en uno de los tres ejes del espacio (X, Y, Z). Conseguir este formato exacto era un paso clave, ya que es el requisito obligatorio que nos pide la plataforma de Inteligencia Artificial (Edge Impulse) para poder tragar los datos a través del puerto serie.
- En el Plotter de Thonny: Al activar la herramienta del Plotter (la gráfica integrada de Thonny), el programa empezó a dibujar tres líneas de colores en tiempo real que representaban los tres ejes. Al dar pequeños golpes en la mesa o mover el sensor con la mano, las líneas reaccionaban al instante creando ondas y picos muy definidos.

Esta prueba me sirvió para validar que el sensor es muy preciso y que reacciona perfectamente a las vibraciones. Con el hardware ya testado y respondiendo en condiciones, el sistema está listo para conectarse a Edge Impulse y empezar a grabar las muestras de movimiento en la cama.

Fase 3: captura y segmentación de datos para el entrenamiento:

Una vez que configuré la Raspberry Pi Pico para que leyera el acelerómetro y guardara el código como `main.py`, logré que la placa funcionara de forma autónoma. Al desconectarla y volverla a conectar, ejecutaba el código automáticamente sin necesidad de abrir Thonny, dejando libre el puerto serie.

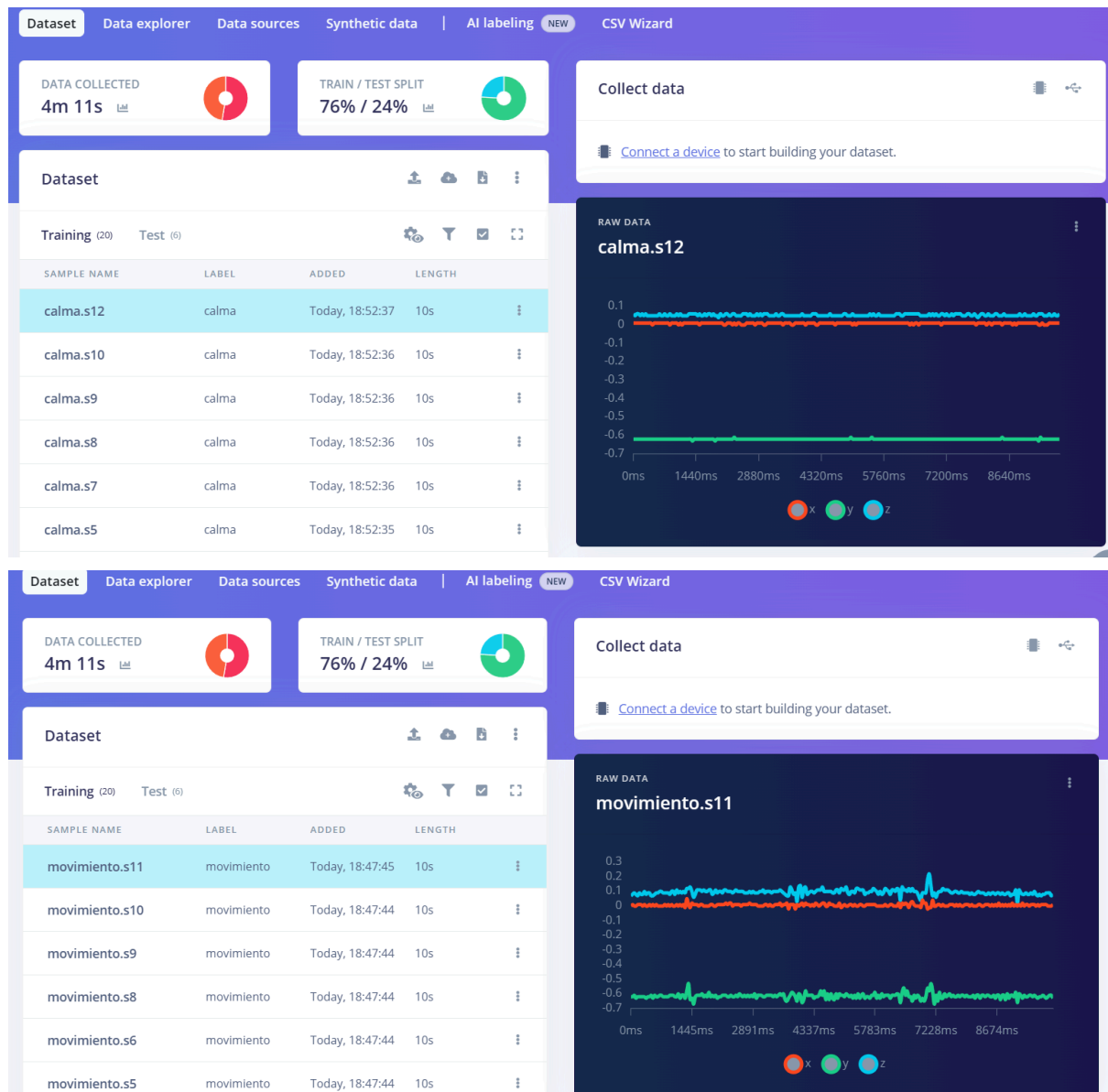
A continuación, para guardar estos datos en el ordenador, utilicé la consola de comandos de Windows (CMD) y ejecuté un script de Python que creé llamado `captura.py`. Este script se conectaba a la Pico a través del puerto COM9, recibía los datos de movimiento en tiempo real y los grababa directamente en archivos de texto con formato CSV. Me coloqué el sensor y realicé dos grabaciones distintas directamente en la cama:



En primer lugar, creé el archivo `calma.csv` manteniéndome completamente inmóvil durante algo más de dos minutos. De esta forma, capturé la firma gráfica de referencia cuando no hay movimiento.

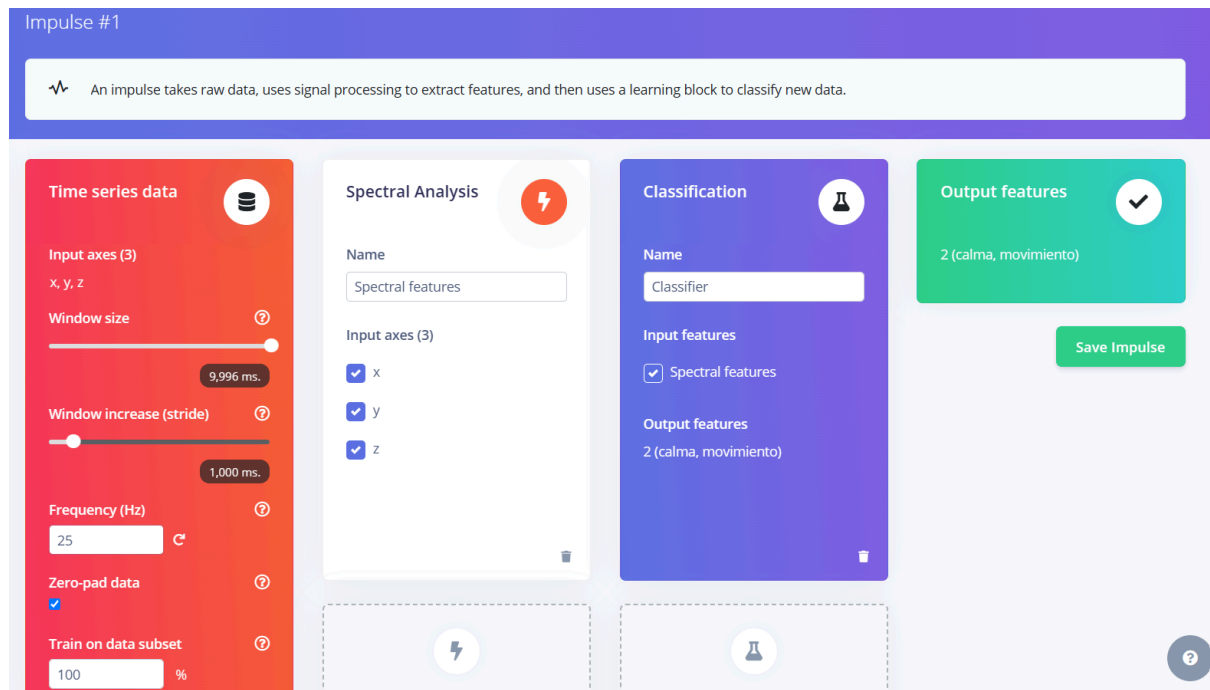
Después, cambié el nombre del archivo de salida en el script de captura y volví a realizar la grabación, esta vez moviéndome, girándome y simulando los movimientos típicos de cuando te estás acomodando o estás en un sueño ligero. Estos datos se guardaron en el archivo `movimiento.csv`.

Por último, cargué ambos archivos CSV en la plataforma web de Edge Impulse. Como las grabaciones eran largas, utilicé las herramientas de segmentación automática de la plataforma. Configuré el sistema para que cortara los archivos de forma continua en bloques más pequeños de 10 segundos cada uno y les asignara la etiqueta correspondiente (`calma` o `movimiento`). Tras este proceso, he conseguido tener un conjunto de datos perfectamente repartido y listo, con un total de 4 minutos y 11 segundos de muestras reales segmentadas en el panel de control.



Fase 4: Diseño de la Arquitectura del Modelo (Impulse Design):

Una vez que los datos estuvieron correctamente cargados y organizados, pasé a la fase de diseño de la estructura de procesamiento del modelo, lo que en la plataforma se conoce como Impulse. En esta etapa configuré un flujo de trabajo compuesto por cuatro bloques principales, conectados en serie para transformar los datos brutos del sensor en una predicción final:



Bloque de datos de entrada (Time Series Data): En este bloque configuré cómo debe leer la plataforma las señales temporales del acelerómetro. Definí el tamaño de la ventana (Window size) en 9.996 milisegundos (aproximadamente los 10 segundos de nuestras muestras) y un incremento de ventana (Window increase o stride) de 1.000 milisegundos. Esto le permite al sistema analizar los datos de los tres ejes (X, Y, Z) solapando las ventanas segundo a segundo para no perder ningún patrón de movimiento. La frecuencia quedó fijada de forma correcta en los 25 Hz del muestreo original.

Bloque de procesamiento de señal (Spectral Analysis): Añadí este bloque específico para extraer las características clave de las gráficas. Su función principal es transformar los datos de aceleración en bruto del sensor y analizar sus frecuencias y ritmos. En lugar de procesar miles de números aislados, este bloque filtra el ruido y resalta la amplitud y la potencia de los movimientos en los tres ejes, facilitando la tarea del clasificador.

Bloque de aprendizaje (Classification): Incorporé un bloque clasificador basado en redes neuronales. Su objetivo es tomar todas las firmas de frecuencia extraídas por el bloque espectral y aprender a asociarlas con sus etiquetas correspondientes a través del entrenamiento.

Bloque de salida (Output Features): El flujo termina en este bloque, que muestra los dos resultados o clases que el sistema es capaz de predecir basándose en el análisis anterior: calma y movimiento.

Finalmente, una vez estructurado todo el flujo de bloques de izquierda a derecha, guardé la configuración del impulso para proceder a la extracción definitiva de características y al entrenamiento de la red neuronal.

Fase 5: Extracción de Características y Explorador de Datos

Una vez guardada la estructura del impulso, pasé a configurar y ejecutar el bloque de procesamiento de señal. En la pestaña de parámetros, mantuve los valores base y procedí a la generación de características espectrales (Spectral Features) para procesar el conjunto de datos de entrenamiento, compuesto por un total de 3 minutos y 11 segundos distribuidos de manera equitativa en 20 ventanas independientes (10 de calma y 10 de movimiento).

The screenshot displays a web-based interface for generating spectral features. It includes a configuration section with the following parameters:

- Data in training set: 3m 11s
- Classes: 2 (calma, movimiento)
- Training windows: 20
- Calculate feature importance: ☐
- Normalize features [?]: Don't normalize data (dropdown menu)

A blue button labeled "Generate features" is positioned to the right of the configuration options.

Below the configuration section, the "Feature generation output" panel shows the execution log:

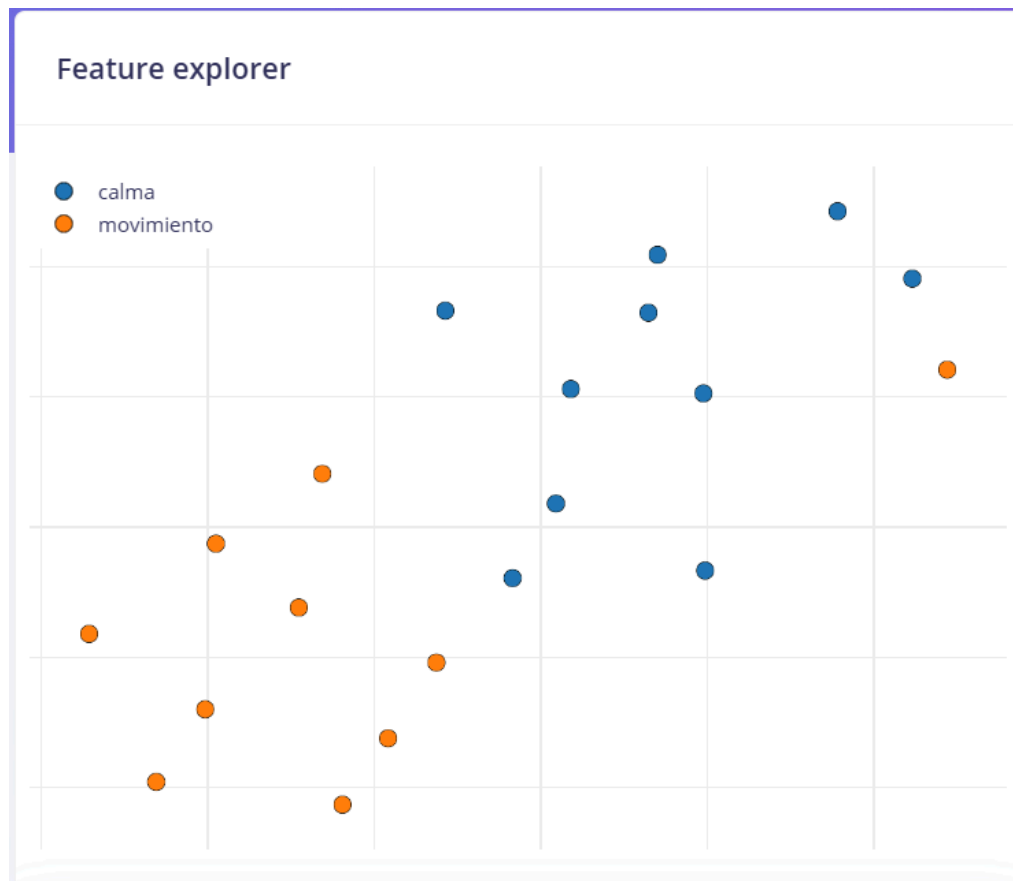
```
✓Job scheduled at 18 Jun 2026 18:19:23
✓Job started at 18 Jun 2026 18:19:26
[ 0/20] Creating windows from files...
[ 1/20] Creating windows from files...
[20/20] Creating windows from files...

Created 20 windows: calma: 10, movimiento: 10

Creating features...
[ 1/20] Creating features...
[20/20] Creating features...
Created features

Job completed (success)
```

La ejecución del proceso concluyó con éxito, generando de manera automática las firmas de frecuencia e intensidad de cada fragmento analizado. El resultado de este procesamiento matemático se visualiza de forma directa en la herramienta de exploración de características (Feature Explorer), un espacio bidimensional en el que el sistema representa gráficamente cada bloque analizado en función de sus componentes espectrales:



Los puntos de color azul representan las muestras etiquetadas como calma. Se observa de forma clara cómo se agrupan firmemente en el cuadrante superior derecho de la gráfica, lo que refleja un comportamiento constante y de muy baja variación energética.

Los puntos de color naranja corresponden a las muestras de movimiento. Estos datos se sitúan de manera muy marcada en la zona inferior izquierda de la pantalla, dejando un claro margen visual respecto a la clase opuesta.

La clara separación física entre ambos grupos de puntos demuestra que el análisis espectral ha sido capaz de diferenciar perfectamente las señales del sensor sin mezclar las clases. Esto confirma que la calidad de los datos de captura reales tomados en la cama es alta y que el modelo de clasificación final contará con patrones geométricos muy claros para aprender a predecir con alta precisión.

Fase 6: Entrenamiento y Resultados del Modelo de Clasificación

Para iniciar el proceso de aprendizaje, accedí a la sección del clasificador dentro de la plataforma con el fin de configurar los parámetros de la red neuronal y ejecutar el entrenamiento definitivo con las muestras de datos obtenidas en la cama.

Neural Network settings



Training settings

Number of training cycles ?	100
Use learned optimizer ?	☐
Learning rate ?	0.005
Training processor ?	CPU

En este panel ajusté los parámetros de optimización para mejorar la calidad del entrenamiento. Elevé el número de ciclos de entrenamiento (Number of training cycles) a 100, permitiendo que la red neuronal procesara de manera iterativa las muestras suficientes veces para consolidar el aprendizaje. Asimismo, fijé la tasa de aprendizaje (Learning rate) en 0.005, un valor adecuado para equilibrar la velocidad de convergencia y la estabilidad del modelo. El proceso se ejecutó utilizando el procesador local asignado por defecto (CPU).

Training output

🔇 (0)

```
9: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
Calculating int8 accuracy...
/app/keras/.venv/lib/python3.10/site-packages/sklearn/metrics/_ranking.py:37
9: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined in that case.
  warnings.warn(
Extracting TensorBoard logs...
Extracting TensorBoard logs OK

Model training complete

Job completed (success)
```

Esta captura muestra el registro del sistema durante la ejecución del algoritmo de aprendizaje. El bloque detalla que el proceso finalizó de manera correcta con la indicación Job completed (success) y generó la confirmación Model training complete. Durante el volcado final, la consola reportó advertencias automáticas comunes debido a que el conjunto de validación en pasos intermedios específicos evaluaba clases de forma independiente, pero procedió sin interrupciones críticas completando la extracción de registros en TensorBoard y el cálculo de la precisión final en formato optimizado de 8 bits (int8).

Model

Model version: ?

Quantized (int8) ▾

Last training performance (validation set)



ACCURACY
100.0%



LOSS
0.01

Confusion matrix (validation set)

	CALMA	MOVIMIENTO
CALMA	100%	0%
MOVIMIENTO	0%	100%
F1 SCORE	1.00	1.00

La última imagen detalla las métricas de rendimiento obtenidas por el modelo tras el entrenamiento completo. El sistema alcanzó una precisión (Accuracy) del 100.0% y un nivel de pérdida (Loss) mínimo de 0.01. Estos resultados quedan respaldados por la matriz de confusión del conjunto de validación, la cual muestra que el 100% de los fragmentos correspondientes a calma fueron clasificados correctamente, y de igual manera, el 100% de los estados de movimiento se identificaron sin errores ni falsos positivos, obteniendo un F1 Score perfecto de 1.00 para ambas clases.

Fase 7: Programación

Funcionamiento de main.py (En la Raspberry Pi Pico)

El programa arranca configurando el pin 16 para controlar el altavoz con frecuencias de sonido, pero empieza con el volumen totalmente apagado para evitar pitidos molestos al conectarlo. Justo después, intenta enlazarse con el chip del acelerómetro mediante el bus de datos I2C y le envía una instrucción directa para sacarlo de su modo de reposo y empezar a medir. En un ciclo continuo que nunca se detiene, lee la memoria interna del sensor, transforma esos bytes en números decimales tradicionales para los ejes X, Y, Z y los imprime por el puerto serie para que el ordenador los reciba. Si detecta que el ordenador envía la orden de encender la alarma, el código activa un temporizador que cada medio segundo enciende el sonido y al medio segundo siguiente lo apaga, imitando el ritmo de un despertador real. En cada vuelta de este ciclo revisa el pin interno de la placa conectado al botón físico; si detecta que lo presionas mientras la alarma suena, corta la corriente del altavoz y le envía un texto de aviso al ordenador para confirmar el apagado.

Funcionamiento de despertador.py (En el Ordenador)

El programa del ordenador abre el puerto USB asignado a la placa, limpia los datos viejos que puedan saturar la memoria de Windows y espera a que la conexión sea estable. Una vez listo, entra en un bucle donde lee constantemente las coordenadas que le envía la Pico por el cable. Con estos números, calcula la magnitud total del movimiento sumando los valores absolutos de los tres ejes, lo que revela la intensidad de la agitación del dispositivo en cada instante. Al mismo tiempo, el script revisa la hora real de tu sistema; si el reloj entra en el tramo programado entre las 7:30 y las 8:00 y nota que la lectura supera el límite de sensibilidad establecido, decide que es momento de despertar al usuario. Si el reloj alcanza la hora límite de las 8:00 y todavía no se ha registrado actividad, el programa activa la alarma por pura seguridad. En cualquiera de los dos casos, el ordenador le manda la orden escrita a la Pico para que empiece a pitar y bloquea el estado del script para que la alarma se quede sonando de forma fija aunque te vuelvas a quedar quieto en la cama.

Resultados obtenidos

El despertador inteligente se ha implementado y validado con éxito, logrando una comunicación bidireccional estable y en tiempo real entre la Raspberry Pi Pico y el ordenador. Los ensayos muestran que el dispositivo es capaz de monitorizar de forma continua la aceleración en los tres ejes espaciales, consolidando las lecturas en un único valor de magnitud total que refleja con precisión cualquier intento de manipulación física.

El control horario automatizado funciona correctamente bajo las ventanas programadas, permitiendo que el software actúe de manera preventiva. Cuando el sistema registra un valor de agitación que supera el umbral establecido entre las 7:30 y las 8:00, o bien cuando se alcanza la hora límite sin actividad, el ordenador envía la orden de activación al microcontrolador de forma instantánea. El buzzer responde de inmediato con una alarma intermitente a una frecuencia de 1000Hz, la cual se mantiene fija bloqueando nuevas lecturas de movimiento. Finalmente, se ha verificado la eficacia del mecanismo de seguridad nativo, logrando que la alarma solo se apague tras una acción consciente del usuario al presionar el botón físico integrado en la propia placa.

Problemas encontrados y cómo se resolvieron

Durante el desarrollo del proyecto, el principal obstáculo técnico fue la imposibilidad de desplegar el modelo de inteligencia artificial diseñado inicialmente, debido a restricciones de compatibilidad para instalar la librería de Edge Impulse en el entorno de Python local. Ante esta limitación crítica, se descartó el enfoque de aprendizaje automático y se reestructuró por completo la lógica del sistema hacia un algoritmo basado en umbrales físicos puros. En lugar de procesar los datos con IA, se programó el ordenador para calcular la magnitud geométrica total sumando los valores absolutos de los tres ejes del acelerómetro, una

solución matemática mucho más ligera, robusta y libre de dependencias externas que detecta el movimiento con total precisión.

Otro desafío técnico importante surgió al constatar que la Raspberry Pi Pico carece de un módulo de reloj en tiempo real (RTC) independiente con batería propia. Al no disponer de este componente de hardware, el microcontrolador es incapaz de mantener la hora correcta por sí mismo y se reinicia por defecto a las 00:00 cada vez que se desconecta de la corriente eléctrica. Para solucionar esta limitación física sin necesidad de añadir costes ni cableado extra al proyecto, se diseñó una arquitectura de software dividida. Se delegó el control horario completo al ordenador mediante un segundo programa ejecutable (despertador.py), el cual aprovecha el reloj interno del sistema operativo para verificar las ventanas de tiempo y enviar de forma externa las órdenes de activación a la placa a través del puerto serie.

Conclusiones

El desarrollo de este proyecto permite concluir que se ha logrado diseñar un sistema de alarma funcional que cumple con el objetivo técnico de comunicarse en tiempo real y reaccionar al movimiento. Sin embargo, el resultado final deja un sabor agridulce debido a las limitaciones de software y hardware encontradas en el camino.

El verdadero propósito del dispositivo era monitorizar el movimiento para despertarte exclusivamente en la fase de sueño ligero. Por ello, la imposibilidad de instalar la librería de Edge Impulse en el entorno local supuso un contratiempo importante. Nos hubiera gustado mucho utilizar el modelo de inteligencia artificial que ya estaba diseñado, ya que conllevó bastante trabajo de desarrollo y habría aportado el análisis inteligente y preciso que requería el proyecto. Al tener que sustituir la IA por una suma matemática de los tres ejes del acelerómetro, el sistema se volvió mucho más simple de lo planeado inicialmente.

Por otra parte, la falta de un módulo de reloj físico en la Raspberry Pi Pico restó comodidad al prototipo, obligándonos a depender de un programa secundario en el ordenador para controlar la hora. En definitiva, aunque el proyecto se ha sacado adelante con éxito y la integración entre los scripts y los componentes funciona perfectamente, las restricciones tecnológicas nos han impedido alcanzar el nivel de optimización e independencia que teníamos en mente para un despertador inteligente ideal.

Bibliografía

<https://pip-assets.raspberrypi.com/categories/610-raspberry-pi-pico/documents/RP-008307-DS-1-pico-datasheet.pdf> - Datasheet Raspberry Pi Pico

<https://www.st.com/resource/en/datasheet/lis3dh.pdf> - Datasheet Acelerómetro LIS3DH

https://www.mouser.com/datasheet/2/744/Seeed_107020000-1217511.pdf - Datasheet Buzzer v1.2